



MOBILNE APLIKACIJE

Vežbe 9

Senzori i multimedija

2025/2026

Sadržaj

1. Senzori	3
1.1. Sensor Framework	3
1.2. Sortiranje objekata pomoću senzora	10
2. Snimanje zvuka	13
3. Reprodukcijska zvuka	17
4. Snimanje fotografije i video zapisa	18
5. Primeri	20

1. Senzori

Senzor je uređaj koji pretvara jednu fizičku veličinu u drugu fizičku veličinu, koju čovek može neposredno da opazi ili koju računar može da očita.

Android podržava tri tipa senzora:

1. Senzori pozicije
2. Senzori pokreta
3. Senzori okruženja

Senzori pozicije mere fizičku poziciju uređaja. Ova kategorija uključuje senzore za orijentaciju i magnetometre (MAGNETIC_FIELD, PROXIMITY).

Senzori pokreta mere sile ubrzanja i rotacione sile duž tri ose. Senzori pokreta su: akcelerometri, gravitacioni senzori, žiroskopi i senzor rotacionog vektora (ACCELEROMETER, GRAVITY, GYROSCOPE, LINEAR_ACCELERATION, ROTATION_VECTOR).

Senzori okruženja mere različite parametre okoline, kao što su temperatura, pritisak, osvetljenje, vlaga. Ova kategorija uključuje barometre, fotometre i termometre (AMBIENT_TEMPERATURE, LIGHT, PRESSURE, RELATIVE_HUMIDITY).

Hardverski senzori: ACCELEROMETER, AMBIENT_TEMPERATURE, GYROSCOPE, LIGHT, MAGNETIC_FIELD, PRESSURE, PROXIMITY, RELATIVE_HUMIDITY

Softverski ili hardverski senzori: GRAVITY, LINEAR_ACCELERATION, ROTATION_VECTOR

1.1. Sensor Framework

Sensor Framework omogućava pristup različitim tipovima senzora i rad sa njima. *Sensor Framework* je deo paketa *android.hardware* i uključuje sledeće klase i interfejse:

- *SensorManager*,
- *Sensor*,
- *SensorEvent* i
- *SensorEventListener*.

SensorManager

Ova klasa omogućava različite metode za pristupanje, izlistavanje senzora itd.

Sensor

Klasa *Sensor* može da se koristi da se kreira instanca nekog specifičnog senzora. Ova klasa sadrži informacije o svojstvima određenog senzora i nudi niz metoda koje omogućavaju da se ustanove mogućnosti senzora.

SensorEvent

Sistem koristi klasu *SensorEvent* da kreira sensor event objekat, koji sadrži informacije o određenom merenju.

SensorEventListener

Interfejs *SensorEventListener* služi za primanje notifikacija od *SensorManager*-a. On sadrži obrađivače *SensorEvent* događaja.

Sa *Sensor Framework*-om možemo da:

- Odredimo koji senzori su dostupni na uređaju.
- Odredimo mogućnosti dostupnih senzora.
- Napišemo obrađivače događaja, koji reaguju na promenu fizičke veličine ili tačnosti merenja.
- Registrujemo ili izbrišemo obrađivače događaja, koji prate promene u merenju.

U primeru za vežbe 9 kreirali smo klasu *SensorsFragment* koja implementira *SensorEventListener* (slika 1).

```
public class SensorsFragment extends Fragment implements SensorEventListener {  
  
    14 usages  
    private FragmentSensorsBinding binding;  
    8 usages  
    private SensorManager sensorManager;  
    2 usages  
    private Sensor accelerometer;  
    2 usages  
    private Sensor gyroscope;  
    2 usages  
    private Sensor magneticField;  
    2 usages  
    private Sensor proximity;  
    2 usages  
    private Sensor linearAcceleration;  
}
```

Slika 1. Kreiranje klase koja implementira interfejs *SensorEventListener*

Da bi se ustanovilo koji sve senzori postoje na uređaju, potrebno je dobiti referencu do sensor servisa (slika 2). Kreira se instanca klase *SensorManager* pozivanjem metode *getSystemService* i prosleđivanjem konstante *SENSOR_SERVICE* (linija 26). Nad objektom *sensorManager*-a pozivaćemo metode da registrujemo *listener*-e, da bismo dobijali merenja.

Listu svih senzora možemo dobiti uz pomoć metode *getSensorList()*. Toj metodi prosleđujemo konstantu *TYPE_ALL*. Primer:

```
List<Sensor> deviceSensors = sensorManager.getSensorList(Sensor.TYPE_ALL);
```

Za dobavljanje senzora određenog tipa umesto *TYPE_ALL* prosleđuje se *TYPE_GYROSCOPE* ili *TYPE_GRAVITY* ili *TYPE_ACCELERATION* itd.

Osim što možemo da dobavimo listu senzora, moguće je koristiti *public* metode klase *Sensor* da dobavimo informacije o svojstvima određenog senzora. Nemaju svi uređaji iste vrste senzora, niti ista vrsta senzora na različitim uređajima ima jednaka svojstva. Metode koje klasa *Sensor* nudi mogu biti veoma korisne, ako želimo da se naša aplikacija ponaša različito u zavisnosti od tipova senzora i njihovih svojstava na različitim uređajima. Neke od metoda koje mogu da se koriste su *getResolution()*, *getMaximumRange()*, *getPower()* itd.

Na slici 2 dobavljamo tekstualna polja sa našeg *layout*-a u koja ćemo upisivati vrednosti merenja. Slika 3 prikazuje *CardView* jednog tipa senzora. *fragment_sensors.xml layout* sadrži po jedno tekstualno polje za svaki senzor koji ćemo pratiti.

```
public void onCreateView(@NonNull View view, @Nullable Bundle savedInstanceState) {  
  
    super.onCreateView(view, savedInstanceState);  
  
    sensorManager = (SensorManager) requireContext()  
        .getSystemService(Context.SENSOR_SERVICE);  
  
    accelerometer = sensorManager.getDefaultSensor(Sensor.TYPE_ACCELEROMETER);  
    gyroscope = sensorManager.getDefaultSensor(Sensor.TYPE_GYROSCOPE);  
    magneticField = sensorManager.getDefaultSensor(Sensor.TYPE_MAGNETIC_FIELD);  
    proximity = sensorManager.getDefaultSensor(Sensor.TYPE_PROXIMITY);  
    linearAcceleration = sensorManager.getDefaultSensor(Sensor.TYPE_LINEAR_ACCELERATION);  
}
```

Slika 2. Metoda *onViewCreated*

```

<androidx.cardview.widget.CardView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginBottom="12dp"
    app:cardCornerRadius="12dp"
    app:cardElevation="6dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:padding="16dp">

        <TextView
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Accelerometer"
            android:textStyle="bold"
            android:textSize="18sp"
            android:textColor="#4F46E5"/>

        <TextView
            android:id="@+id/tvAccelerometer"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:paddingTop="6dp"
            android:textColor="#111827"/>
    </LinearLayout>

</androidx.cardview.widget.CardView>

```

Slika 3. *fragment_sensors.xml*

U primeru za ove vežbe pratimo merenja 5 senzora:

1. ACCELEROMETER
2. LINEAR_ACCELERATION
3. MAGNETIC_FIELD
4. PROXIMITY
5. GYROSCOPE

U metodi *onResume* registrujemo *listener*-e na senzore da bismo dobijali merenja. *Listener*-e registrujemo uz pomoć metode *registerListener*. Ovu metodu pozivamo nad objektom klase *SensorManager* i prosleđujemo joj tri parametra:

1. *Listener* – obrađivač događaja
2. *Sensor* – je senzor
3. *SamplingPeriodUs* – predstavlja period uzorkovanja

Na slici 4 se nalazi primer registrovanja *listener*-a.

```
public void onResume() {
    super.onResume();
    registerSensor(accelerometer);
    registerSensor(gyroscope);
    registerSensor(magneticField);
    registerSensor(proximity);
    registerSensor(linearAcceleration);
}
5 usages
private void registerSensor(Sensor sensor) {
    if (sensor != null) {
        sensorManager.registerListener( listener: this, sensor, SensorManager.SENSOR_DELAY_GAME);
    }
}
```

Slika 4. Metoda *onResume*

Interfejs *SensorEventListener* nam nudi 2 metode koje implementiramo:

1. *onSensorChanged*
2. *onAccuracyChanged*

onSensorChanged

Metoda *onSensorChanged* se poziva kada se vrednosti merenja promene. Ova metoda prima *event*, objekat klase *SensorEvent*, koji sadrži informacije o merenju. Kod na slici 5, u zavisnosti od tipa senzora, popunjava tekstualna polja *layout*-a. U svako polje unosi se *string* sa nazivom senzora, koji se prati, i sa pročitanim vrednostima. Vrednosti čitamo iz niza *values*, koji sadrži objekat *event*.

```

public void onSensorChanged(SensorEvent event) {

    int type = event.sensor.getType();

    float x = event.values[0];
    float y = event.values.length > 1 ? event.values[1] : 0;
    float z = event.values.length > 2 ? event.values[2] : 0;

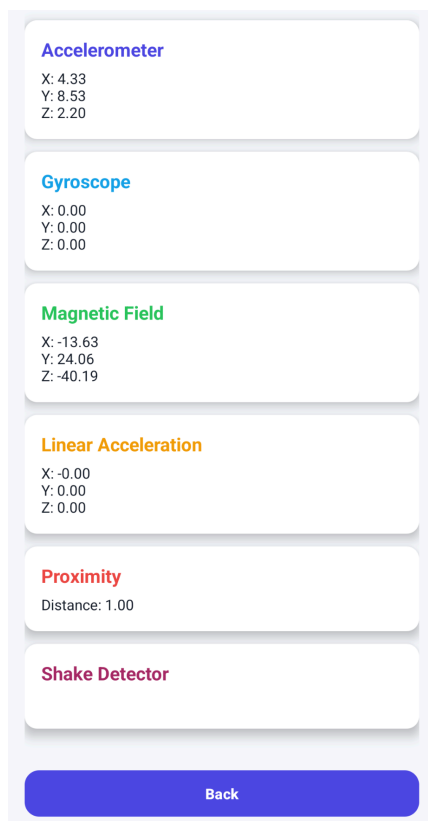
    String values = String.format(Locale.getDefault(), format: "X: %.2f\nY: %.2f\nZ: %.2f", x, y, z);

    if (type == Sensor.TYPE_ACCELEROMETER) {
        binding.tvGyroscope.setText(values);
    } else if (type == Sensor.TYPE_MAGNETIC_FIELD) {
        binding.tvMagneticField.setText(values);
    } else if (type == Sensor.TYPE_LINEAR_ACCELERATION) {
        binding.tvLinearAcceleration.setText(values);
    } else if (type == Sensor.TYPE_PROXIMITY) {
        binding.tvProximity.setText(
            String.format(Locale.getDefault(), format: "Distance: %.2f", x)
        );
    }
}
}

```

Slika 5. Metoda *onSensorChanged*

Na slici 6 se nalazi primer kako se merenja prikazuju u našoj aplikaciji.



Slika 6. Prikaz merenja

onAccuracyChanged

Metoda *onAccuracyChanged* se poziva kada se promeni tačnost merenja registrovanog senzora. Za razliku od metode *onSensorChanged*, ova metoda se poziva samo kad se tačnost promeni.

Kada više ne koristimo senzore, sve *listener*-e treba otkaćiti i to radimo u metodi *onPause* (slika 7).

```
public void onPause() {  
    super.onPause();  
    sensorManager.unregisterListener(this);  
}
```

Slika 7. Metoda *onPause*

1.2. Shake event

Za registrovanje *shake* događaja (slika 9) koristimo *sensorManager* koji će nam služiti za praćenje promena od strane *Accelerometer* senzora. U zavisnosti od promena vrednosti po x, y, i z osama, beležimo *shake* događaj i ispisujemo u tekstualnom polju.

Unutar metode *onSensorChanged* preuzimamo x, y i z vrednosti svakih 500ms i proračunavamo brzinu (slika 10). Ukoliko je brzina veća od definisanog praga, registrujemo *shake* događaj.

```
if (type == Sensor.TYPE_ACCELEROMETER) {
    binding.tvAccelerometer.setText(values);
    long currentTime = System.currentTimeMillis();

    if (!initialized) {
        lastX = x;
        lastY = y;
        lastZ = z;
        initialized = true;
        return;
    }

    float deltaX = x - lastX;
    float deltaY = y - lastY;
    float deltaZ = z - lastZ;
    lastX = x;
    lastY = y;
    lastZ = z;

    float shakeForce = (float) Math.sqrt(deltaX * deltaX + deltaY * deltaY + deltaZ * deltaZ);
    if (shakeForce > SHAKE_THRESHOLD && currentTime - lastShakeTime > 500) {

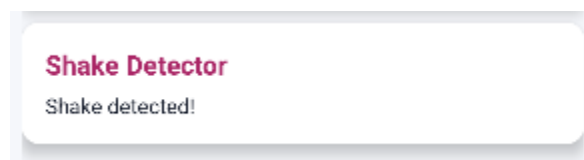
        lastShakeTime = currentTime;
        Log.i( tag: "SHAKE", msg: "DETECTED");
        binding.tvShake.setText("Shake detected!");

        binding.tvShake.setAlpha(0f);
        binding.tvShake.animate()
            .alpha( value: 1f)
            .setDuration(200)
            .start();

        binding.tvShake.postDelayed(() -> {
            binding.tvShake.setText("Shake not detected!");
        }, delayMillis: 1000);
    }
}
```

Slika 8. Izračunavanje vrednosti za *shake* događaj

Da bi se detekcija događaja videla na prikazu aplikacije, unutar kartice u *layout*-u ispisujemo "Shake detected!" (slika 9).



Slika 9. Reagovanje na *shake* događaj

2. Snimanje zvuka

Android platforma omogućava snimanje zvuka posredstvom mikrofona. Snimanje zvuka je moguće samo na fizičkim uređajima, a emulator nema tu mogućnost.

MediaRecorder je API za snimanje različitih audio i video formata.

Da bismo snimili zvuk potrebno je da izvršimo sledeće korake:

- Instancirati *MediaRecorder*.
- Postaviti audio izvor korišćenjem *setAudioSource* metode.
- Postaviti izlazni format korišćenjem *setOutputFormat* metode.
- Postaviti ime izlazne datoteke korišćenjem *setOutputFile* metode.
- Postaviti audio koder korišćenjem *setAudioEncoder* metode.
- Pozvati *prepare* metodu.
- Pozvati *start* metodu za početak snimanja zvuka.
- Pozvati *stop* metodu za završetak snimanja zvuka.
- Osloboditi instancu *MediaRecorder*-a pozivanjem *release* metode.

Za demonstriranje ovog primera kreirali smo klasu *AudioRecordingFragment*. U njenoj *onCreateView* metodi postavljamo naš layout *fragment_audio_recording.xml*. U metodi *onViewCreated* postavljamo dugmad i tražimo da se odobri pravo pristupa za snimanje audio zapisa (slika 10). Ako korisnik ne odobri pravo pristupa, prikazuje mu se *Toast* poruka i onemogućava se pristup fragmentu. U suprotnom poziva se metoda *updateUiState()* za inicijalizaciju elemenata neophodnih za snimanje i reprodukciju zvuka (slika 11).

```

@Nullable
@Override
public View onCreateView(@NonNull LayoutInflater inflater, @Nullable ViewGroup container, @Nullable Bundle savedInstanceState) {
    binding = FragmentAudioRecordingBinding.inflate(inflater, container, attachToParent: false);
    return binding.getRoot();
}

@Override
public void onViewCreated(@NonNull View view, @Nullable Bundle savedInstanceState) {
    super.onViewCreated(view, savedInstanceState);
    outputFile = Objects.requireNonNull(requireContext()).getExternalCacheDir().getAbsolutePath() + "/audio_record.mp4";
    binding.backButton.setOnClickListener( View v ->
        requireActivity().getSupportFragmentManager().popBackStack()
    );
    setupButtons();
    requestPermission();
}

```

Slika 10. Metode *onCreateView()* i *onViewCreated()*

```

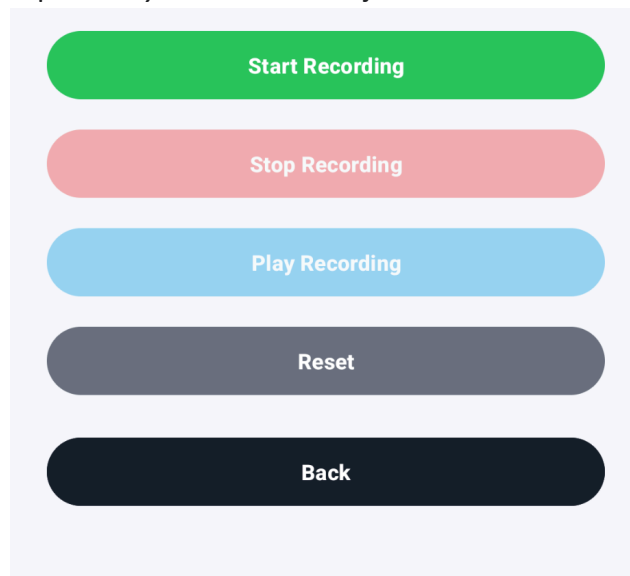
private void updateUiState(State state) {

    switch (state) {
        case IDLE:
            setEnabled(binding.startButton, enabled: true);
            setEnabled(binding.stopButton, enabled: false);
            setEnabled(binding.playButton, enabled: false);
            setEnabled(binding.resetButton, enabled: true);
            break;
        case RECORDING:
            setEnabled(binding.startButton, enabled: false);
            setEnabled(binding.stopButton, enabled: true);
            setEnabled(binding.playButton, enabled: false);
            setEnabled(binding.resetButton, enabled: false);
            break;
        case READY:
            setEnabled(binding.startButton, enabled: true);
            setEnabled(binding.stopButton, enabled: false);
            setEnabled(binding.playButton, enabled: true);
            setEnabled(binding.resetButton, enabled: true);
            break;
    }
}

```

Slika 11. Metoda *updateUiState()*

Na slici 12 se može videti prikaz *layout*-a za snimanje zvuka.



Slika 12. Prikaz fragmenta za audio

Unutar metode *setupButtons()* dobavljamo dugmad za snimanje, stopiranje, reprodukovanje i resetovanje zvučnog zapisa, a zatim registrujemo *listener*-e za svaki od njih (slika 13).

```
private void setupButtons() {  
    updateUiState(State.IDLE);  
    binding.startButton.setOnClickListener( View v -> startRecording());  
    binding.stopButton.setOnClickListener( View v -> stopRecording());  
    binding.playButton.setOnClickListener( View v -> playRecording());  
    binding.resetButton.setOnClickListener( View v -> resetRecording());  
}
```

Slika 13. Inicijalizacija elemenata za snimanje i reprodukciju zvučnog zapisa

U *AndroidManifest* datoteci definisali smo i statička prava pristupa (slika 14) za snimanje zvuka (RECORD_AUDIO).

```
<uses-permission android:name="android.permission.RECORD_AUDIO" />
```

Slika 14. Definisanje statičkih prava pristupa

Metoda *start* se poziva kada korisnik klikne na dugme *start*. U ovoj metodi (slika 15) nad objektom klase *MediaRecorder* pozivamo metode:

- *setAudioSource* - postavljamo audio izvor.
- *setOutputFormat* - postavljamo izlazni format.
- *setAudioEncoder* - postavljamo audio koder.
- *setOutputFile* - postavljamo ime izlazne datoteke.

Inicijalizaciju završavamo pozivom metode *prepare*, nakon čega pozivamo metodu *startRecording* za početak snimanja zvuka. Na samom kraju metode ažurirami UI čime omogućujemo korisniku da klikne na stop i reset dugme, dok ostala zabranjujemo.

```
private void startRecording() {  
  
    try {  
        releaseRecorder();  
  
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {  
            mediaRecorder = new MediaRecorder(requireContext());  
        } else {  
            mediaRecorder = new MediaRecorder();  
        }  
  
        mediaRecorder.setAudioSource(MediaRecorder.AudioSource.MIC);  
        mediaRecorder.setOutputFormat(MediaRecorder.OutputFormat.MPEG_4);  
        mediaRecorder.setAudioEncoder(MediaRecorder.AudioEncoder.AAC);  
        mediaRecorder.setOutputFile(outputFile);  
  
        mediaRecorder.prepare();  
        mediaRecorder.start();  
        isRecording = true;  
  
        updateUiState(State.RECORDING);  
        toast(text: "Recording started");  
  
    } catch (Exception e) {  
  
        e.printStackTrace();  
        toast(text: "Start recording failed");  
  
        isRecording = false;  
        updateUiState(State.IDLE);  
  
    }  
}
```

Slika 15. Metoda *start*

Kada korisnik klikne na dugme *stop*, poziva se *stopRecording* metoda, koja oslobađa instancu *MediaRecorder*-a pozivanjem *release* metode (slika 16).

```
private void stopRecording() {  
  
    if (!isRecording || mediaRecorder == null) return;  
  
    try {  
        mediaRecorder.stop();  
    } catch (RuntimeException e) {  
        toast(text: "Recording corrupted");  
    } finally {  
        releaseRecorder();  
        isRecording = false;  
    }  
  
    updateUiState(State.READY);  
    toast(text: "Recording saved");  
}
```

Slika 16. Metoda *stopRecording()*

3. Reprodukcijska zvuka

Android platforma omogućava reprodukciju audio i video sadržaja u različitim formatima i preko različitih mrežnih protokola. Moguće je reprodukovati sadržaj iz datoteka koje su skladištene kao resursi, datoteka u internom ili eksternom skladištu podataka ili iz toka podataka, koji stiže preko mreže.

Klase koje koristimo za puštanje audio ili video sadržaja su:

- *MediaPlayer*
- *AudioManager*

MediaPlayer je API za reprodukciju audio i video sadržaja.

AudioManager upravlja audio izvorima i audio izlazom.

Korisnik može da reprodukuje audio zapis koji je prethodno snimio. Metoda *playRecording* se poziva kada korisnik klikne na dugme *play* (slika 17). U toj metodi kreira se objekat klase *MediaPlayer* i postavlja mu se putanja do zvuka koji treba da se pusti, pozivanjem metode *setDataSource*. Kao i kod *MediaRecorder*-a, sledi pozivanje metode *prepare* i *start*.

```
private void playRecording() {  
  
    try {  
        stopPlayer();  
  
        mediaPlayer = new MediaPlayer();  
        mediaPlayer.setDataSource(outputFile);  
        mediaPlayer.prepare();  
        mediaPlayer.start();  
  
        toast( text: "Playing audio");  
  
        mediaPlayer.setOnCompletionListener( MediaPlayer mp -> stopPlayer());  
    } catch (IOException e) {  
        e.printStackTrace();  
        toast( text: "Playback failed");  
    }  
}
```

Slika 17. Reprodukcijska zvuka

4. Snimanje fotografije i video zapisa

Snimanje fotografija i video zapisa je moguće korišćenjem postojeće aplikacije (*Camera*). Kao i kod snimanja zvuka, koristimo *MediaRecorder*. Za snimanje fotografija i videa korišćenjem *MediaRecorder* API-a potrebno je izvršiti sledeće korake:

- Detektovati kameru.
- Pristupiti kameri.
- Napraviti *Preview* klasu.
- Napraviti *Preview* raspored.
- Podesiti obrađivače događaja.
- Snimiti fotografiju ili video u datoteku.
- Osloboditi kameru.

U *AndroidManifest* datoteci dodajemo novo pravo pristupa (CAMERA) - slika 18.

```
<uses-feature
    android:name="android.hardware.camera"
    android:required="false" />

<uses-permission android:name="android.permission.CAMERA" />
```

Slika 18. Definisanje statičkih prava pristupa za kameru

Unutar klase *PictureCaptureFragment*, dodajemo *imageView* za pregled slike i dugme *takePictureButton*. Potom vršimo proveru da li su permisije odobrene i ako nisu tražimo da se odobre. U slučaju da permisija nije omogućena, dugme postavljamo na *disabled* (slika 19).

```
public void onCreate(@Nullable Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);

    cameraLauncher = registerForActivityResult(
        new ActivityResultContracts.StartActivityForResult(),
        ActivityResult result -> {
            if (result.getResultCode() == android.app.Activity.RESULT_OK) {
                if (binding != null && imageUri != null) {
                    binding.imageView.setImageURI(imageUri);
                    toast(msg, "Image captured");
                }
            }
        }
    );

    permissionLauncher = registerForActivityResult(new ActivityResultContracts.RequestPermission(), Boolean isGranted -> {
        if (isGranted) {
            openCamera();
        } else {
            binding.takePictureButton.setEnabled(false);
            toast(msg, "Camera permission required");
        }
    });
}
```

Slika 19. Inicijalizacija elemenata za snimanje fotografije i proveru permisija

Kada korisnik želi da uslika potrebno je da klikne na dugme *Take a picture*, koje će pozvati metodu *openCamera* (slika 20). U toj metodi instanciramo nameru, startujemo aktivnost i primamo rezultat od aktivnosti.

GenericFileProvider je pomoćna klasa koja nasleđuje *FileProvider*. *FileProvider* predstavlja specijalnu potklasu *ContentProvider*-a koja nam omogućava sigurnije rukovanje fajlovima tako što kreira *content://URI* za datoteku umesto *file:///URI*. *Content URI* omogućava rad sa datotekama korišćenjem privremenih permisija.

EXTRA_OUTPUT je *URI* koji određuje putanju i ime datoteke u koju će se sačuvati fotografija (ili video).

Ovo je primer dobijanja slike u punoj rezoluciji kamere.

```
private void openCamera() {

    Intent intent = new Intent(MediaStore.ACTION_IMAGE_CAPTURE);

    File imageFile = createImageFile();
    if (imageFile == null) {
        toast(msg: "File error");
        return;
    }

    imageUrl = FileProvider.getUriForFile(requireContext(), authority: requireContext().getPackageName() + ".provider", imageFile);
    intent.putExtra(MediaStore.EXTRA_OUTPUT, imageUrl);
    intent.addFlags(Intent.FLAG_GRANT_READ_URI_PERMISSION | Intent.FLAG_GRANT_WRITE_URI_PERMISSION);

    cameraLauncher.launch(intent);
}
```

Slika 20. Metoda *openCamera()*

Android *Camera* aplikacija može da sačuva fotografije ako joj postavimo putanju na kojoj treba da ih čuva. Fotografije, koje korisnik napravi, mogu se čuvati na različitim putanjama, tako da svaka aplikacija ili samo aplikacija koja je kreirala sliku može da im pristupi. U svrhu generisanja putanje, napravljena je pomoćna metoda *createImageFile()*.

Odgovarajući direktorijum za smeštanje fotografija se dobija pozivanjem metode *getExternalFilesDir* i prosleđivanjem konstante *DIRECTORY_PICTURES* (slika 21). Pošto naziv datoteke (putanja) mora biti jedinstven, na naziv direktorijuma dodajemo datum i vreme kada je fotografija napravljena.

```
private File createImageFile() {

    String timeStamp = new SimpleDateFormat( pattern: "yyyyMMdd_HHmmss", Locale.getDefault()).format(new Date());
    File storageDir = requireContext().getExternalFilesDir(Environment.DIRECTORY_PICTURES);
    if (storageDir == null) return null;
    return new File(storageDir, child: "IMG_" + timeStamp + ".jpg");
}
```

Slika 21. Pomoćna metoda *createImageFile()*

U *AndroidManifest* datoteci definisali smo i *FileProvider* (slika 22), a prethodno i neophodna prava pristupa.

```
<provider
    android:name=".providers.GenericFileProvider"
    android:authorities="${applicationId}.provider"
    android:exported="false"
    android:grantUriPermissions="true">
    <meta-data
        android:name="android.support.FILE_PROVIDER_PATHS"
        android:resource="@xml/file_paths" />
</provider>
```

Slika 22. Definisanje *FileProvider*-a u *AndroidManifest* datoteci

5. Primeri

Domaći se nalazi na *Canvas-u* (*canvas.ftn.uns.ac.rs*) na putanji *Вежбе/09 Задамак.pdf*.

Primer možete preuzeti na sledećem linku:

<https://gitlab.com/mobilne-aplikacije/mobilne-aplikacije/>

Za dodatna pitanja možete se obratiti asistentima:

- Milan Jovkić (milan.jovkic@uns.ac.rs)
- Jelena Matković (matkovic.jelena@uns.ac.rs)